Reviewer Pack — Minimal Rank of Kac-Moody Algebras Bounds Communication Comp…

Entry 523a719847a3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 19:54:42 UTC

Minimal Rank of Kac-Moody Algebras Bounds Communication Complexity for Read-Twice BP

Entry ID: `523a719847a3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 19:54:42 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Kac-Moody Algebras) **Field B** (complexity object): Complexity Theory: Communication Complexity (Read-twice BP)

Statement:

> ['For any read-twice branching program P with size s, the minimal rank of its associated Kac-Moody algebra is Θ(s).', 'For any communication protocol C that requires t bits for read-twice BP, there exists a Kac-Moody algebra of rank at most O(t^2).', 'This invariant provides an upper bound on communication complexity in terms of the rank of the associated algebra.']

Rationale (proposer's reasoning):

> ['Kac-Moody algebras are known to capture certain algebraic structures that arise from representation theory, which may provide a deeper understanding of the computational resources needed for read-twice BP.', 'Previous work has shown connections between algebraic geometry and communication complexity, suggesting that algebraic tools could be applied to study communication problems.', 'This conjecture proposes an explicit link between the rank of Kac-Moody algebras and communication complexity, which could lead to new algorithms or lower bounds.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1a08a787b2274997`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all read-twice BP instances P with size s, the minimal rank of the associated Kac-Moody algebra is within a factor of 1.5 from s and no protocol C requiring t bits has an associated Kac-Moody algebra with a rank greater than O(t^2). The conjecture is falsified if any instance P has a Kac-Moody algebra's minimal rank outside

this range or if there exists a protocol C with t bits that requires an associated Kac-Moody algebra of rank greater than O(t^2).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_add(A, B):
    m = len(A)
    n = len(A[0])
    result = [[A[i][j] + B[i][j] for j in range(n)] for i in range(m)]
    return result

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(cols)] for i, row in enumerate
```

```
    for i in range(rows):
        pivot_row = max(range(i, rows), key=lambda r: abs(augmented_matrix[r][i]))
        augmented_matrix[i], augmented_matrix[pivot_row] = augmented_matrix[pivot_row], augmented_

        if augmented_matrix[i][i] == 0:
            raise ValueError("Singular matrix")

        for j in range(i + 1, rows):
            factor = -augmented_matrix[j][i] / augmented_matrix[i][i]
            augmented_matrix[j] = [factor * x + y for x, y in zip(augmented_matrix[i], augmented_m

    rank = sum(1 for row in augmented_matrix if any(x != 0 for x in row[:cols]))
    return rank

def generate_bp_instance(n):
    bp_instance = [random.choice([0, 1]) for _ in range(n)]
    return bp_instance + bp_instance[::-1]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):
            bp_instance = generate_bp_instance(n)
            kac_moody_rank = rank([[bp_instance[i], bp_instance[n + i]] for i in range(n)])

            if kac_moody_rank == 0:
                continue

            instances_tested += 1
            total_metric_value += kac_moody_rank

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = all(0.5 * n <= mean_metric_value <= 1.5 * n for n in n_values)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Kac-Moody Rank",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
```

```
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{trial_result}...}}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65cd697b.py", line 104, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65cd697b.py", line 73, in run_trial
    kac_moody_rank = rank([[bp_instance[i], bp_instance[n + i]] for i in range(n)])
                     ^^^^
NameError: name 'rank' is not defined. Did you mean: 'range'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to an undefined ‘rank’ variable, which means that the conjecture could not be tested according to the pre-registered support condition. | next: Investigate and define the ‘rank’ function properly in the code. Once corrected, re-run the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12165
2	preregistration	ollama_remote	glm4:latest	0	0	7127
3	novelty	ollama_remote	glm4:latest	0	0	5739
4	novelty	ollama_remote	glm4:latest	0	0	5238
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15326
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10608
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12740
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12961
9	judge	ollama_remote	glm4:latest	0	0	8457

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 90360 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/523a719847a3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/523a719847a3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/523a719847a3.tar.gz` (if generated)